

Green-Light Security for AI Computing

— Toward Secure, Energy-Efficient, and Sustainable AI Infrastructure

Gianna Zhang

Abstract

All computing systems face a tension between security and performance. This tension is most acute in AI computing, where unprecedented speed requirements meet high-consequence risk: failures in protection can inflict unusually large financial, operational, and societal damage by compromising models, leaking sensitive data, and disrupting critical services. **Green-Light Security for AI** shows how AI can operate at full speed with safety on. We reorganize protection to align with how AI runs: keeping light, continuous safeguards in place and calling on heavier measures only when truly needed. We support this approach with rigorous mathematical formulations and analysis, together with a compact hardware demo that demonstrates practicality. In short, this project shows AI systems can keep firm guardrails and full speed, the light stays green.

1 Introduction

AI services are expanding so rapidly that cost and energy have become first-order constraints. Global investment in AI-capable data centers is projected to exceed \$5 trillion by 2030, and their electrical demand will reach on the order of ~ 100 gigawatts (GW)¹. For society to capture the full benefit of the AI era, computing infrastructure must remain fully utilized on productive workloads rather than on defensive overhead. Yet the very security that keeps models and data safe imposes a steady protection tax: the constant resource cost of staying secure. This tax includes extra bytes traveling with each data block, additional checks before releasing results, and continuous power drawn by protection logic. At scale, such persistent overhead aggregates into substantial capacity loss and fleet-wide slowdowns.

We address this challenge with a design framework, *Green-Light Security*, that lowers the protection tax while preserving strong security. The key idea is to separate what must happen all the time from what is only needed occasionally. Lightweight verification runs continuously so normal operation remains fast. Heavier corrective work, such as error correction or re-authentication, runs only when a fault is detected. This maintains end-to-end integrity while preventing protective tasks from dominating bandwidth and energy use. When compression is available, it can be used opportunistically to strengthen Green-Light Security rather than to save storage. The small and predictable space created by compression can hold authentication and integrity information, which lets protection travel with the data without increasing overall traffic or power consumption.

All design choices are guided by a quantitative model that links security parameters to latency, cost, and energy. The model ensures that confidentiality, integrity, and reliability targets are met with minimal overhead. This manuscript first develops the architectural principles and the mathematical formulation, and then presents empirical evaluation and prototypes.

¹One GW can power about one million homes in the United States.

2 Motivation and Background

2.1 The Protection Tax in AI Memory Systems

Modern AI accelerators such as GPUs (graphics processing units) and TPUs (tensor processing units) rely on HBM (high-bandwidth memory) to move large volumes of model weights and key-value caches between compute and memory. In these architectures, performance is frequently limited by data movement rather than by arithmetic computation [1, 2, 3]. To ensure correctness and security, systems attach protection metadata that consumes both bandwidth and storage. A common configuration includes an ECC (error-correcting code) to detect and correct bit errors [4, 5] and a MAC (message authentication code) [6] to verify that data has not been tampered with. Although essential, these mechanisms impose a consistent capacity and bandwidth overhead that acts as a *protection tax* on every memory operation.

Quantifying the protection tax. Let a memory transaction carry C_{data} bytes of payload and C_{ECC} and C_{MAC} bytes of protection metadata for reliability and integrity. We define the *protection tax* as the per-transaction overhead relative to useful payload:

$$\tau = \frac{C_{\text{ECC}} + C_{\text{MAC}}}{C_{\text{data}}}. \quad (1)$$

A common illustrative baseline uses a 64B cache line with 16B ECC and 8B MAC, which yields $\tau = (16 + 8)/64 = 37.5\%$. Parameters vary across systems: stronger integrity may require a longer MAC tag, and different memories may allocate different ECC redundancy. Algebraically, any increase in C_{ECC} or C_{MAC} raises τ linearly. It is convenient to express the share of capacity and bandwidth available to the application as the *payload efficiency*

$$\eta = \frac{C_{\text{data}}}{C_{\text{data}} + C_{\text{ECC}} + C_{\text{MAC}}} = \frac{1}{1 + \tau}. \quad (2)$$

Only the fraction η of each transfer carries useful data, while the remaining fraction $1 - \eta$ is consumed by protection metadata, $\eta = 0.73$. Thus an accelerator with 200GB of HBM and 6TB/s of raw bandwidth effectively delivers $0.73 \times 200 = 146\text{GB}$ of usable capacity and $0.73 \times 6 = 4.4\text{TB/s}$ of usable bandwidth to the application.

Why the tax persists. Integrity mechanisms such as MACs must be verified before releasing plaintext to software, which enforces verify-before-decrypt semantics in authenticated designs [6]. Likewise, ECC must be read and checked on every access to detect and, when needed, correct silent errors [4, 5]. Because both checks occur on every transaction, the protection tax applies uniformly to all the reads and writes rather than only to rare events. As model sizes and continuous inference workloads grow, this steady per-access cost becomes an important limiter in AI computing infrastructure, where overall performance is frequently constrained by data movement [1, 2, 3].

Implication. At fleet scale, even modest values of τ compound into measurable slowdowns and higher energy per inference. The central objective of Green-Light Security is therefore to maintain strong reliability and integrity while reducing τ as seen by the application, so that a larger fraction of each HBM transfer carries useful payload rather than paying protection tax.

2.2 Block Size vs. Protection Overhead

The protection tax as discussed above is not only a function of protection strength; it is also a function of protection granularity. Traditional systems attach ECC and MAC to small 64B cache lines, duplicating metadata many times and inflating τ . By contrast, AI accelerators naturally move data in larger blocks (e.g., hundreds or thousands of bytes) so it is both natural and safe to aggregate protection over a coarser unit. Intuitively, when a fixed amount of metadata protects a larger payload, the fractional overhead per byte drops. The next paragraphs make this intuition precise, starting with reliability (ECC) and then integrity (MAC). Below, we quantify this effect for error correction and authentication.

ECC overhead. Modern computing systems commonly employ Reed–Solomon (RS) codes for memory protection, as they provide strong multi-symbol correction capability with modest redundancy. An (n, k) RS code encodes k data symbols into n total symbols over a Galois field $\text{GF}(2^m)$ with m bits per symbol, adding $(n - k)$ parity symbols that can correct up to $t = \lfloor (n - k)/2 \rfloor$ symbol errors. Given a raw bit-error rate p_b and assuming independent, uniformly distributed bit errors, the per-symbol error rate is

$$p_s = 1 - (1 - p_b)^m. \quad (3)$$

The decoding-failure probability equals the probability that more than t symbols in the codeword are corrupted, i.e., the upper tail of a $\text{Binomial}(n, p_s)$ distribution:

$$P_{\text{fail}}(n, t, p_s) = \sum_{i=t+1}^n \binom{n}{i} p_s^i (1 - p_s)^{n-i}. \quad (4)$$

For a fixed reliability target $P_{\text{fail}} \leq \varepsilon$ and the same raw p_s , increasing the *protected payload size* k (i.e., applying a single RS codeword over a larger data block) allows the ECC overhead $\tau_{\text{ECC}} = (n - k)/k$ to decrease while still meeting ε . For demonstration, consider $p_b = 1 \times 10^{-5}$ and $\varepsilon = 1 \times 10^{-18}$. Table 1 lists the required ECC redundancy $(n - k)$ and the resulting τ_{ECC} for several user-data sizes k per RS codeword. As k increases from 128B to 256B, the field changes from $\text{GF}(2^8)$ to $\text{GF}(2^{16})$.

Table 1: ECC overhead vs. block size.

m	8	8	16	16	16	16	16
k (bytes)	64	128	256	512	1024	2048	4096
$n - k$ (bytes)	16	18	28	44	52	60	72
τ_{ECC}	25.00%	14.06%	10.94%	8.59%	5.08%	2.93%	1.76%

As the user-data block protected by a single RS codeword grows (larger k), the minimum parity needed to satisfy $P_{\text{fail}} \leq \varepsilon$ typically increases much more slowly than k . Consequently, the overhead $\tau_{\text{ECC}} = (n - k)/k$ falls monotonically with k . This trend is clear in Table 1: moving from $k = 64\text{B}$ to $k = 4096\text{B}$ reduces τ_{ECC} from 25.00% to 1.76% despite a larger absolute parity budget, reinforcing the observation that, when latency and implementation constraints allow, increasing k is an effective means of reducing ECC overhead.

MAC overhead. MACs detect unauthorized modification of a protected data unit. Let the baseline protection granularity be n m -bit symbols with a symbol-aligned tag of L_{MAC} symbols. Under ideal assumptions, the forgery (mis-detection) probability per n -symbol unit is

$$p = 2^{-m \cdot L_{\text{MAC}}}. \quad (5)$$

Now, instead of authenticating every n -symbol unit separately, group s contiguous n -symbol units into a single protection domain of size $s \cdot n$ symbols, and authenticate that domain with one tag of length L'_{MAC} symbols. To preserve the original per- n -symbol risk, i.e., to keep the probability that *any one* of the s constituent n -symbol sub-blocks is tampered with and goes undetected at most p , a conservative union bound gives

$$2^{-m \cdot L'_{\text{MAC}}} \leq \frac{p}{s} = \frac{2^{-m \cdot L_{\text{MAC}}}}{s}, \quad (6)$$

which implies

$$m \cdot L'_{\text{MAC}} \geq m \cdot L_{\text{MAC}} + \log_2 s \quad \Rightarrow \quad L'_{\text{MAC}} \geq L_{\text{MAC}} + \left\lceil \frac{\log_2 s}{m} \right\rceil, \quad (7)$$

where the ceiling enforces byte alignment. Thus the tag length grows only *logarithmically* in s , while payload grows *linearly* in s . The resulting fractional MAC overhead for the coarser granularity is

$$\tau'_{\text{MAC}} = \frac{L'_{\text{MAC}}}{s \cdot n}, \quad (8)$$

so τ'_{MAC} decreases nearly inversely with s even after compensating for a slightly longer MAC tag. For example, with $L_{\text{MAC}}=8$ at $n=64$ and $m = 8$, aggregating to $s=16$ (i.e., protecting 1024B per tag) requires $L'_{\text{MAC}} = 8 + \left\lceil \frac{\log_2 16}{8} \right\rceil = 9$ symbols to preserve the per- n -symbol mis-detection probability. More generally, $s \in [2, 256]$ needs only one extra byte, while $s \in [257, 65536]$ needs two extra bytes, illustrating that integrity overhead per byte falls rapidly as protection granularity increases.

3 Green-Light Security

Green-Light Security reorganizes protection mechanisms across the memory hierarchy to minimize the steady protection tax while preserving full reliability and integrity. In this section, we first present the proposed architecture, which decouples reliability and integrity across memory tiers and enlarges the protection granularity to reduce per-byte overhead. We then develop the corresponding mathematical formulations to quantify these trade-offs and establish design rules that balance efficiency, reliability, and cost.

3.1 Key Design Principles

Rather than disabling safeguards, the architecture keeps all checks active but relocates where and how they operate. The core principle is to distinguish between protection that must accompany every data access and protection that can safely be invoked on demand. Today’s AI accelerators treat reliability and integrity in separate layers: Each DRAM die inside HBM stacks performs internal ECC encoding and decoding transparently, while the AI accelerator attaches a MAC tag before the data leaves its trusted boundary. As illustrated in Fig. 1(a), HBM stores complete ECC codewords that each cover both the user data and its MAC tag. On every access, the HBM device retrieves the full codeword, performs error detection (and correction if needed), and returns the corrected user data plus MAC tag to the AI accelerator for authentication. This design ensures correctness but forces every transaction to pay a steady capacity and bandwidth cost for both ECC parity and MAC tags, even when no errors occur.

As illustrated in Fig. 1(b), Green-Light Security preserves end-to-end guarantees while reorganizing where and how protection is applied. Reliability and integrity reside in distinct memory tiers aligned with their access frequency. Exploiting AI workload access patterns, we deliberately increase the protection granularity so that more user data share the same metadata, reducing the steady protection tax without compromising correctness. This reorganization is driven by the observation established in Section 2.2: when a fixed-strength authenticator is amortized over m n -byte units, the required tag length grows only logarithmically in m , so the per-byte integrity overhead falls nearly as $1/m$; likewise, when RS codewords protect a larger user-data payload k , the parity fraction $\tau_{\text{ECC}} = (n - k)/k$ declines monotonically for a fixed decoding-failure target ε . Consequently, aggregating protection to match the naturally coarse transfer sizes of AI workloads reduces the steady protection tax while preserving the same end-to-end integrity and reliability targets. The rest of this subsection details how integrity remains in-path via a coarse-grained MAC and how reliability moves off-path via RS parity, while preserving end-to-end security guarantees.

Primary MAC (in-path integrity). Each user-data block in HBM is protected by a coarser-grained MAC tag. Instead of authenticating every small cache line, several contiguous cache lines are grouped into a single protection domain. As derived in Section 2.2, enlarging the domain amortizes the tag over a larger payload, so the per-byte integrity overhead reduces. The MAC travels with the data and is verified on every access, preserving verify-before-use semantics with negligible latency.

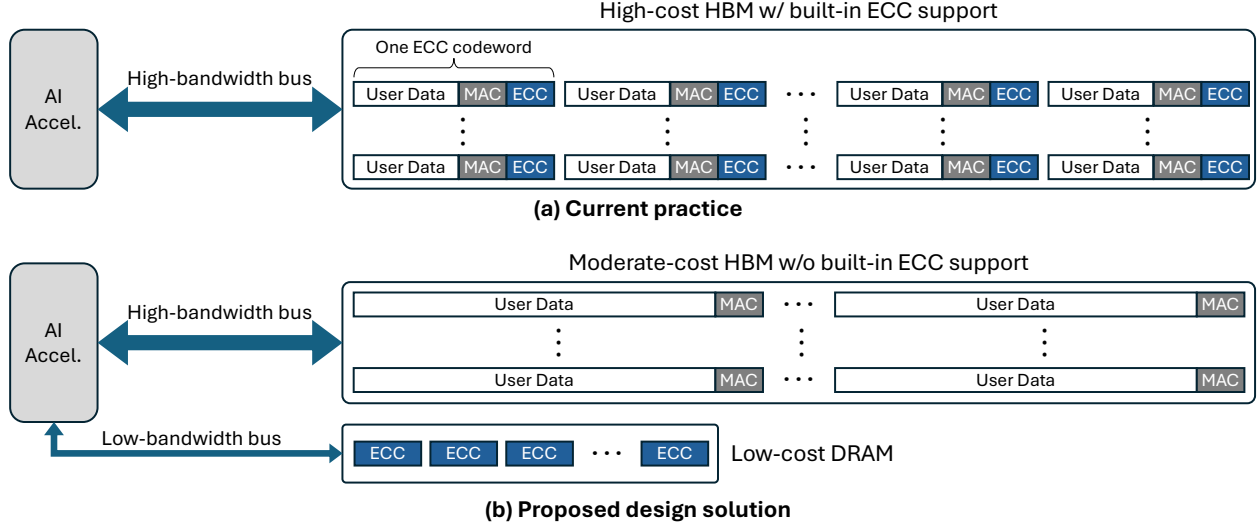
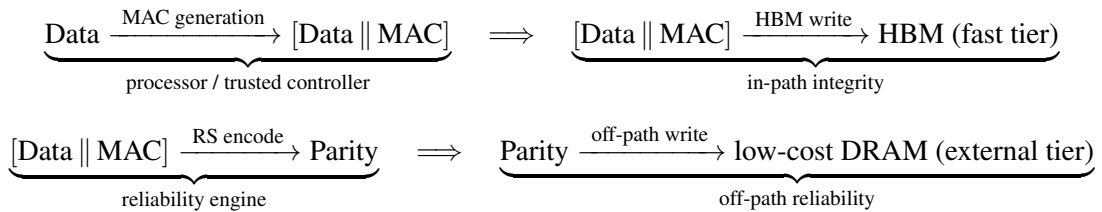


Figure 1: (a) Current practice: each HBM die performs internal ECC encoding and decoding, storing complete codewords that cover both user data and its MAC tag. (b) Proposed design: integrity remains in-path with a coarse-grained MAC stored in HBM, while reliability is provided off-path by ECC parity stored in low-cost external DRAM, eliminating on-die ECC and reducing HBM cost.

ECC parity (off-path reliability). RS code parity for the *combined* block (user data || MAC) is computed over the same enlarged data region but *stored off-path* in a lower-cost external DRAM tier. Because codewords operate on larger block size, the ECC overhead τ_{ECC} decreases while meeting the same target reliability. ECC parity is fetched from the low-cost DRAM only when correction is required, removing persistent ECC traffic from the high-bandwidth HBM. Because the vast majority of HBM reads succeed without ECC intervention, the off-path parity fetch is a rare, latency-tolerant event. The critical path for common reads remains the on-path MAC verification, which adds negligible delay (no more than a few nanoseconds).

The external parity store is treated as untrusted. On reads, HBM first verifies the MAC. If verification fails, the AI accelerator fetches the corresponding ECC parity and performs RS decoding to reconstruct (user data || MAC). The block is *accepted* only if the MAC validates after error correction. Any parity tampering that yields an incorrect reconstruction will fail MAC verification and be rejected. Thus data integrity is preserved without an additional off-path MAC.

Protection placement and flow. To further clarify the end-to-end pathway, we spell out the sequence and destinations explicitly as follow (see Fig. 1(b)):



On reads, the AI accelerator first verifies the MAC on $[\text{Data} \parallel \text{MAC}]$ fetched from HBM. Only if verification fails does the AI accelerator fetch off-path ECC parity from the low-cost DRAM, perform RS decoding to reconstruct $[\text{Data} \parallel \text{MAC}]$, and re-verify the MAC before release.

HBM cost implication. Relocating reliability to the external tier removes the requirement for built-in on-die ECC in HBM. The fast tier can therefore use HBM devices without integrated ECC logic, reducing bill-of-materials and controller complexity while keeping integrity checks in-path.

3.2 Mathematical Formulation

This section develops a quantitative model that formalizes the design principles introduced in Section 3.1. Green-Light Security seeks to minimize the effective protection tax τ_{eff} , capturing the joint impact of protection on both memory cost and system speed. The framework integrates reliability, integrity, and throughput into a single formulation, enabling designers to choose protection granularity that is both secure and fast. Table 2 summarizes the key notation and parameters.

Table 2: Notation and parameters used in the mathematical formulation.

Symbol	Definition
m	Number of bits per symbol in ECC codewords
ℓ_{usr}	Number of m -bit symbols in each user data block protected by one ECC codeword
ℓ_{MAC}	Number of m -bit symbols in each MAC tag protected by one ECC codeword
ℓ_{ECC}	Number of m -bit ECC parity symbols in one ECC codeword
ℓ_{cwd}	Total number of m -bit symbols in one ECC codeword ($\ell_{\text{cwd}} = \ell_{\text{usr}} + \ell_{\text{MAC}} + \ell_{\text{ECC}}$)
p_b	Raw bit-error rate in memory
B_{HBM}	Total HBM bandwidth
B_{DRAM}	Off-path (low-cost) DRAM bandwidth for parity fetches
D_{ECC}	Total ECC decoding throughput in AI accelerator
p_{err}	Probability a read triggers ECC decoding
$w_{\text{cap}}, w_{\text{spd}}$	User-configurable weights for cost (w_{cap}) and speed (w_{spd})

Probability of invoking ECC decoding. The raw bit-error rate p_b serves as an environmental input parameter that determines how often faults occur in memory. A read request triggers off-path ECC decoding only when the in-path MAC fails verification; hence the protection granularity influences both the cost of metadata and the frequency of correction events. ECC decoding is invoked when at least one bit in the HBM-resident payload (user data || MAC) is corrupted:

$$p_{\text{err}} = 1 - (1 - p_b)^{m \cdot (\ell_{\text{usr}} + \ell_{\text{MAC}})}. \quad (9)$$

This quantity represents the arrival rate of ECC correction requests. As the HBM-resident payload size $m \cdot (\ell_{\text{usr}} + \ell_{\text{MAC}})$ increases, the probability of invoking ECC decoding p_{err} rises even though the per-byte ECC redundancy decreases.

Reliability. As discussed above, we assume the use of RS code as memory ECC, which has been widely adopted in modern computing systems thanks to its strong correction strength on both random and burst errors. Under uniform independent bit errors, the probability that a symbol of m bits in a RS codeword is corrupted is

$$p_s = 1 - (1 - p_b)^m. \quad (10)$$

Given ℓ_{ECC} parity bytes per RS codeword and m bits per symbol, each RS codeword can correct up to $t = \lfloor \ell_{\text{ECC}}/2 \rfloor$ symbol errors. Hence the RS decoding failure probability can be calculated as

$$P_{\text{fail}} = \sum_{i=t+1}^{\ell_{\text{cwd}}} \binom{\ell_{\text{cwd}}}{i} p_s^i (1 - p_s)^{\ell_{\text{cwd}} - i}, \quad (11)$$

and the reliability constraint requires $P_{\text{fail}} \leq \varepsilon_{\text{rel}}$. Larger payload size ($\ell_{\text{usr}} + \ell_{\text{MAC}}$) decreases ECC overhead but increases the number of bits subject to random faults, raising the chance of correction events.

Integrity. Integrity ensures that any modification to protected data is detected. When ℓ_{MAC} m -bit symbols of tag protect user data, the forgery probability per codeword is

$$P_{\text{forg}} = 2^{-m \cdot \ell_{\text{MAC}}} \leq \varepsilon_{\text{int}}. \quad (12)$$

Coarsening protection (i.e., covering larger data blocks under a single tag) reduces metadata overhead almost inversely with the size of the protection domain, while maintaining cryptographic strength.

Capacity cost. Because MAC metadata resides in HBM while ECC parity is stored off-path in low-cost DRAM, their contributions to the cost overhead are asymmetric. Let $\alpha \in (0, 1)$ denote the normalized bit-cost of external low-cost DRAM relative to HBM. The normalized capacity cost (relative to useful user data ℓ_{usr}) is therefore

$$C_{\text{cap}} = \underbrace{\frac{\ell_{\text{MAC}}}{\ell_{\text{usr}}}}_{\text{HBM-resident MAC}} + \underbrace{\alpha \cdot \frac{\ell_{\text{ECC}}}{\ell_{\text{usr}}}}_{\text{DRAM-resident parity}}. \quad (13)$$

Introducing α makes it explicit that relocating ECC parity to a cheaper tier lowers the dollar cost of protection even when the raw byte overhead is unchanged, and it shifts the optimal system design configurations toward coarser ECC granularity so long as throughput and reliability constraints remain satisfied.

Speed constraint. The sustained effective bandwidth is jointly limited by (i) HBM bandwidth, (ii) the total ECC decoding throughput, and (iii) the low-cost DRAM bandwidth available for parity fetches. Recall that D_{ECC} denotes the achievable ECC decoding throughput of AI accelerators. Given the ECC decoding occurrence probability p_{err} , we can express the decoding-bound throughput as $D_{\text{ECC}}/p_{\text{err}}$. Meanwhile, given the low-cost DRAM bandwidth B_{DRAM} , we can express the parity-fetching-bound throughput as $B_{\text{DRAM}} \cdot (\ell_{\text{usr}} + \ell_{\text{MAC}})/(p_{\text{err}} \cdot \ell_{\text{ECC}})$. Therefore, the sustained effective bandwidth is obtained as

$$B_{\text{eff}} = \min \left\{ B_{\text{HBM}}, \frac{D_{\text{ECC}}}{p_{\text{err}}}, \frac{B_{\text{DRAM}} \cdot (\ell_{\text{usr}} + \ell_{\text{MAC}})}{p_{\text{err}} \cdot \ell_{\text{ECC}}} \right\}. \quad (14)$$

Accordingly, we can calculate the normalized speed performance loss as

$$L_{\text{spd}} = 1 - \frac{B_{\text{eff}}}{B_{\text{HBM}}}. \quad (15)$$

To avoid speed performance loss, we should prevent ECC decoding or ECC parity fetching from throttling the HBM data path, i.e.,

$$D_{\text{ECC}} \geq p_{\text{err}} \cdot B_{\text{HBM}}, \quad B_{\text{DRAM}} \geq \frac{p_{\text{err}} \cdot \ell_{\text{ECC}}}{\ell_{\text{usr}} + \ell_{\text{MAC}}} \cdot B_{\text{HBM}}. \quad (16)$$

These constraints highlight the central tension: coarser granularity reduces C_{cap} but simultaneously raises p_{err} , increasing demand on both the ECC decoding throughput and low-cost DRAM bandwidth.

Weighted objective and optimization. To balance memory cost and speed performance, Green-Light Security defines a tunable scalar objective:

$$J_{\text{weighted}} = w_{\text{cap}} C_{\text{cap}} + w_{\text{spd}} L_{\text{spd}}, \quad w_{\text{cap}} + w_{\text{spd}} = 1. \quad (17)$$

The design problem is

$$\begin{aligned} & \underset{\ell_{\text{usr}}, \ell_{\text{MAC}}}{\text{Minimize}} && J_{\text{weighted}} \\ & \text{Subject to} && \begin{cases} P_{\text{fail}} \leq \varepsilon_{\text{rel}}, \\ P_{\text{forg}} \leq \varepsilon_{\text{int}}. \end{cases} \end{aligned} \quad (18)$$

The weights $(w_{\text{cap}}, w_{\text{spd}})$ express design priorities: a higher w_{cap} favors low-cost configurations, whereas a higher w_{spd} emphasizes throughput.

Discussion. Solving this formulation yields the optimal parameters ℓ_{usr} and ℓ_{MAC} that minimize the total protection tax for a given bit-error rate p_b and system configuration. As the protected block size increases, metadata overheads τ_{MAC} and τ_{ECC} decrease, reducing memory cost. However, the corresponding rise in p_{err} and the limited $D_{\text{ECC}}(n, k)$ and B_{DRAM} eventually cause throughput saturation, forming the boundary of feasible operation. The resulting *green-light zone* defines operating points that satisfy reliability and integrity targets while minimizing both memory overhead and performance loss, achieving the central goal of secure, efficient, and sustainable AI computing.

4 Evaluation

This section evaluates the proposed design solution through comprehensive simulations. The results further reveal how the raw memory bit-error rate, ECC strength, bandwidth ratios, and memory-cost asymmetry influence the effective protection tax. The simulation quantifies the key metrics—reliability, integrity, capacity cost, and speed loss, and identifies the operating region where Green-Light Security minimizes both memory and performance overheads. In this study, we traverse a grid over $(\ell_{\text{usr}}, \ell_{\text{MAC}})$ and, for each grid point, choose the smallest ℓ_{ECC} that satisfies the reliability target $P_{\text{fail}} \leq \varepsilon_{\text{rel}} = 10^{-18}$. Integrity is enforced at $P_{\text{forg}} \leq 10^{-18}$ per 64B cache line. For the RS code, we set the symbol size to $m = 8$ when the codeword length $\ell_{\text{cwd}} = \ell_{\text{usr}} + \ell_{\text{MAC}} + \ell_{\text{ECC}}$ is $< 512\text{B}$, and to $m = 16$ otherwise. Given $(\ell_{\text{usr}}, \ell_{\text{MAC}}, \ell_{\text{ECC}})$, we compute the error-trigger probability $p_{\text{err}}(p_b)$, the dollar-weighted capacity cost C_{cap} with DRAM cost factor α , and the sustained bandwidth B_{eff} and speed loss L_{spd} under the HBM/ECC/DRAM limit. Finally, we evaluate the weighted objective $J_{\text{weighted}} = w_{\text{cap}} C_{\text{cap}} + w_{\text{spd}} L_{\text{spd}}$ and identify the optimal setting subject to the feasibility and security constraints.

Fig. 2(a) shows the ECC overhead $(n-k)/k$ as a function of the protected block size $(\ell_{\text{usr}} + \ell_{\text{MAC}})$ under different raw bit-error rates p_b . As expected, the overhead decreases with increasing block size because a larger payload amortizes the parity needed to satisfy the reliability target $P_{\text{fail}} \leq 10^{-18}$. However, the curve exhibits a small upward inflection near 256B. This occurs because the RS symbol size m switches from 8 to 16 bits when the total codeword length $\ell_{\text{cwd}} = \ell_{\text{usr}} + \ell_{\text{MAC}} + \ell_{\text{ECC}}$ crosses 256B. Moving to $\text{GF}(2^{16})$ increases the symbol size and, for the same reliability target, the minimum parity required, producing a brief local rise before the overhead resumes its monotonic decline at larger block sizes. Fig. 2(b) shows the trigger probability p_{err} versus block size: as the protected bits grow, p_{err} increases, making explicit the trade-off that lowering ECC overhead by enlarging blocks simultaneously raises the likelihood of invoking ECC decoding.

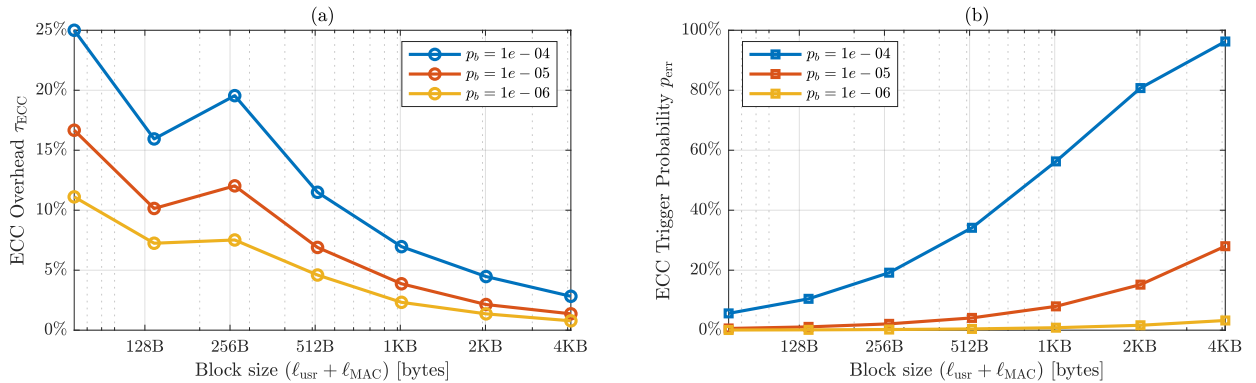


Figure 2: ECC overhead and ECC trigger probability as a function of block size.

Fig. 3 maps the operating regions of the proposed Green-Light Security memory hierarchy as determined by the relative bandwidth and decoding constraints. Each panel sweeps the normalized ratios $B_{\text{DRAM}}/B_{\text{HBM}}$ (x-axis) and $D_{\text{ECC}}/B_{\text{HBM}}$ (y-axis) for different payload sizes ℓ_{usr} and raw bit-error rates

p_b . The colored regions indicate which term in

$$B_{\text{eff}} = \min \left\{ B_{\text{HBM}}, \frac{D_{\text{ECC}}}{p_{\text{err}}}, \frac{B_{\text{DRAM}} \cdot (\ell_{\text{usr}} + \ell_{\text{MAC}})}{p_{\text{err}} \cdot \ell_{\text{ECC}}} \right\}$$

is active: **HBM-bound** (green), **ECC-decoding-bound** (yellow), or **DRAM-bound** (red). The dashed horizontal boundary at $D_{\text{ECC}}/B_{\text{HBM}} = p_{\text{err}}$ and the dashed vertical boundary at

$$\frac{B_{\text{DRAM}}}{B_{\text{HBM}}} = p_{\text{err}} \cdot \frac{\ell_{\text{ECC}}}{\ell_{\text{usr}} + \ell_{\text{MAC}}}$$

separate the regions, where $p_{\text{err}} = 1 - (1 - p_b)^{m(\ell_{\text{usr}} + \ell_{\text{MAC}})}$ is the probability that a read triggers ECC decoding. Increasing ℓ_{usr} (coarser protection granularity) or decreasing p_b shifts operating points toward the HBM-bound regime by amortizing parity and reducing correction frequency; smaller blocks or higher error rates expand the ECC- and DRAM-limited zones. These maps delineate the *green-light* operating region where reliability, integrity, and throughput targets are jointly satisfied with minimal protection tax.

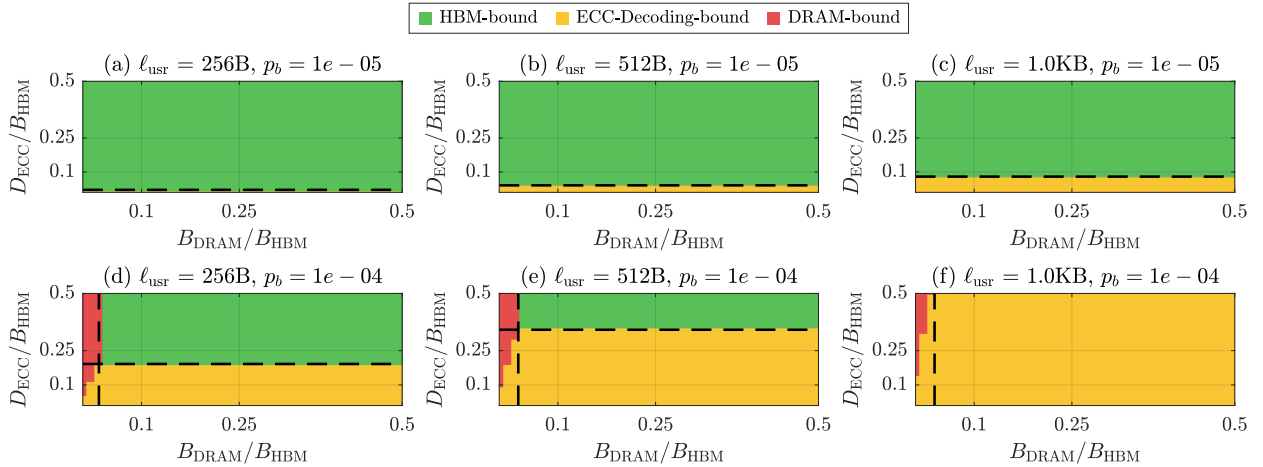


Figure 3: Throughput bottleneck regions under green-light security.

Fig. 4 shows the optimization results obtained by solving Eq. (18) under the configuration $\alpha = 0.1$ (i.e., the bit cost of low-cost DRAM is $10\times$ lower than that of HBM), $\varepsilon_{\text{rel}} = \varepsilon_{\text{int}} = 1 \times 10^{-18}$ (i.e., both the ECC decoding-failure probability and forgery probability must remain below 10^{-18}), and $w_{\text{cap}} = w_{\text{spd}} = 0.50$ (i.e., equal weighting between DRAM cost and speed performance in the optimization objective). These parameters assign balanced importance to cost and performance while maintaining stringent reliability and integrity guarantees. The baseline design protects each 64B cache line with 16B ECC parity and 8B MAC, representing the conventional all-HBM protection scheme. The six panels in Fig. 4 jointly illustrate how the optimal design adapts to environmental and system parameters. The top row shows the *capacity-efficiency gain* (i.e., the ratio of usable capacity under Green-Light Security relative to the baseline), while the bottom row shows the *throughput gain* (i.e., the ratio of speed performance under Green-Light Security relative to the baseline). Because the optimizer dynamically adjusts ℓ_{usr} to ensure operation in the HBM-bound region, the system consistently avoids throughput throttling. As a result, the throughput gain remains fixed at $1.38\times$ across all configurations. This constant value reflects the inherent performance advantage of GLS, which eliminates per-access ECC decoding traffic within HBM and fetches parity only on actual fault events.

In Figs. 4(a)–(b), the raw bit-error rate p_b is swept while holding $B_{\text{DRAM}}/B_{\text{HBM}} = 0.1$ (i.e., the off-path DRAM bandwidth is 10% of the HBM bandwidth) and $D_{\text{ECC}}/B_{\text{HBM}} = 0.3$ (i.e., the ECC decoding throughput is 30% of the HBM bandwidth). As p_b decreases from 10^{-4} to 10^{-6} , the probability of invoking off-path ECC correction drops exponentially. This allows the optimizer to select larger user-data

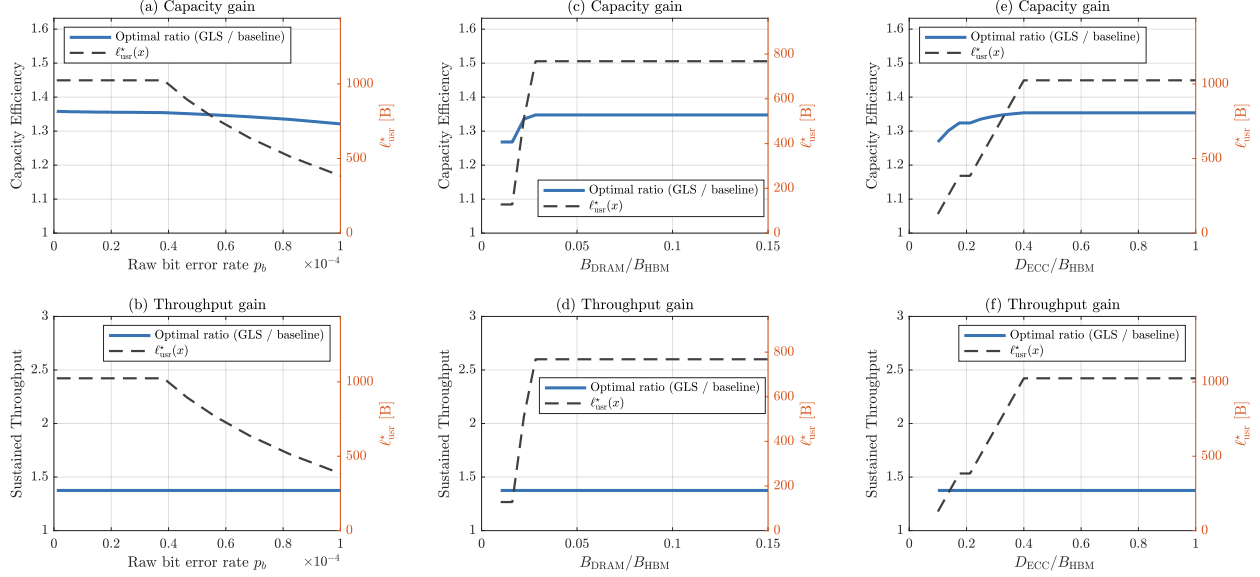


Figure 4: Optimization results of Green-Light Security showing capacity and throughput gains under varying system and reliability parameters.

block sizes ℓ_{usr}^* (from about 500B to 1KB), which amortize MAC and parity overhead. Consequently, the effective payload efficiency increases, yielding approximately 40% capacity improvement over the baseline. Figs. 4(c)–(d) sweep $B_{\text{DRAM}}/B_{\text{HBM}}$ while fixing $p_b = 6 \times 10^{-5}$ and $D_{\text{ECC}}/B_{\text{HBM}} = 0.3$. When DRAM bandwidth is limited ($B_{\text{DRAM}}/B_{\text{HBM}} \leq 0.1$), the optimizer selects smaller blocks (below 200B) to prevent the system from becoming DRAM-bound, resulting in a lower capacity efficiency of about $1.27\times$. As $B_{\text{DRAM}}/B_{\text{HBM}}$ increases beyond 0.1, parity fetches can be served more quickly, allowing ℓ_{usr}^* to increase to roughly 800B and boosting capacity efficiency to $1.35\times$. Figs. 4(e)–(f) sweep $D_{\text{ECC}}/B_{\text{HBM}}$ while fixing $p_b = 6 \times 10^{-5}$ and $B_{\text{DRAM}}/B_{\text{HBM}} = 0.1$. At low decoding throughput ($D_{\text{ECC}}/B_{\text{HBM}} < 0.2$), the optimizer constrains ℓ_{usr}^* to maintain reliability without saturating the ECC decoder. As the available ECC throughput rises, ℓ_{usr}^* increases sharply to around 1KB, achieving greater metadata amortization while sustaining the same throughput ceiling imposed by HBM bandwidth.

Overall, the results in Fig. 4 confirm that the proposed Green-Light Security architecture achieves simultaneous and consistent improvements in both cost and speed efficiency. By relocating ECC parity to low-cost DRAM (weighted by $\alpha = 0.1$) and invoking off-path correction only on demand, Green-Light Security reduces the persistent protection tax that constrains conventional all-HBM solutions. The optimization framework dynamically adjusts the protection granularity ℓ_{usr}^* to sustain HBM-bound operation, yielding a stable $1.38\times$ throughput improvement and 35% capacity-cost reduction. Together, these results demonstrate that Green-Light Security transforms protection from a continuous bandwidth drain into an adaptive, cost-aware, and high-performance mechanism for secure AI computing.

5 Hardware Prototype and Experimental Validation

To complement the analytical and simulation results in Section 4, we constructed a compact hardware prototype that emulates the tiered protection flow of Green-Light Security. The goal of the prototype is not to implement full cryptographic or coding primitives, but rather to validate the architectural separation between fast in-path integrity verification and slow off-path reliability correction, and to demonstrate that the resulting throughput behavior matches the predictions of our model.

5.1 Prototype Architecture

Figure 5 shows the physical realization of the prototype, which consists of a Raspberry Pi representing the fast memory tier and an ESP32 microcontroller acting as the off-path reliability tier. The Raspberry Pi stores user data and coarse-grained MAC tags, performs in-path integrity checks on every access, and completes the vast majority of reads locally. The ESP32, connected via a short USB link, functions as an external reliability store analogous to low-cost DRAM in the proposed architecture of Fig. 1(b). Upon simulated integrity failure, the Pi contacts the ESP32 and receives emulated parity data after a programmable delay. This physical separation mirrors the tiered design in which integrity remains in-path and reliability is invoked only when necessary.

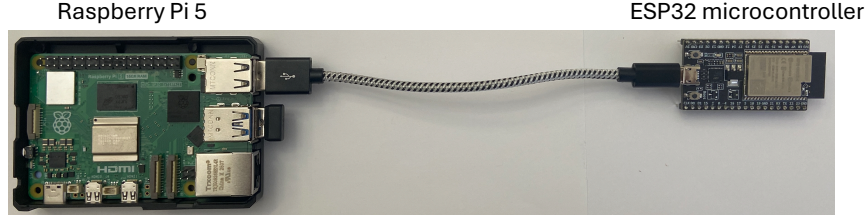


Figure 5: Compact hardware prototype using a Raspberry Pi (fast tier) and ESP32 microcontroller (off-path reliability tier). The ESP32 is accessed only upon simulated MAC failure.

5.2 Emulation and Measurement Strategy

The goal of the prototype is not to implement full cryptographic or coding primitives in hardware, but to measure the real timing behavior of the three components that determine protection cost: in-path integrity verification, local RS decoding, and off-path parity fetch. All latency measurements are performed directly on the Raspberry Pi and ESP32.

Integrity verification is measured by computing a truncated SHA-256 tag on the Pi. Across approximately 200 trials, MAC operations complete in the $1\text{--}3\mu\text{s}$ range, reflecting the extremely low cost of in-path integrity checks. Error correction is measured by decoding RS codewords of various sizes. Using a Python RS library, the Raspberry Pi exhibits decoding latencies of $\sim 1\text{ms}$ for 64B blocks, $\sim 6\text{ms}$ for 512B, $\sim 17\text{ms}$ for 1024B, and $\sim 40\text{ms}$ for 2048B. These measurements reflect the monotonic growth in ECC decoding cost. Off-path RS parity fetching is emulated by sending a short request from the Pi to the ESP32, which responds right away. Across 20 trials, the measured round-trip latency averages roughly 60ms with low variance. This reproduces the large latency gap between fast in-path verification and slow off-path correction assumed by the Green-Light architecture.

To compare these raw measurements with the analytical model of Section 3.2, the prototype automatically converts them into effective architectural latencies. Local latencies are scaled into the millisecond range to approximate realistic pipeline delays in fast-memory controllers to match the ESP32 round-trip latency. A single user-adjustable parameter, the RS scaling factor α_{RS} , uniformly rescales all measured RS decode times.

5.3 Experimental Procedure and Results

For each experiment, the Raspberry Pi issues a stream of read requests. Each request first performs a local MAC verification using the effective in-path latency. The MAC check failure probability is calculated as

$$p_{\text{err}}(B, p_b) = 1 - (1 - p_b)^{8(B+8)}, \quad (19)$$

where B is the block size. If MAC check fails, the Pi triggers off-path correction: it fetches emulated parity from the ESP32 using the remote latency and then performs RS decoding using the scaled cost. This

reproduces the exact control flow of the proposed architecture: fast in-path integrity on every access, and slow off-path reliability only when needed.

Fig. 6 presents the measured-throughput curves for the baseline (64B) protection domain and for Green-Light Security (GLS) with block sizes of 512B, 1024B, and 2048B. At low raw BER, the ECC-trigger probability is negligible. All GLS configurations avoid the slow path and maintain a high, flat throughput plateau, outperforming the baseline and achieving the expected speedup. As the bit error rate increases, larger blocks exhibit an earlier decline in throughput because their larger bit footprint increases p_{err} , causing more frequent off-path corrections. This ordering (2048B falling before 1024B, which falls before 512B) matches the monotonic growth in trigger probability. At high BER (above 10^{-6}), all GLS curves enter the DRAM-bound regime, and throughput decreases accordingly. Despite its simplicity, the prototype reproduces the characteristic behavior of the full design: fast-path stability, rare slow-path invocation in realistic error regimes, and predictable transitions between HBM-bound and DRAM-bound operation. This alignment across analysis, simulation, and hardware measurement demonstrates that Green-Light Security is practical and robust under realistic timing conditions.

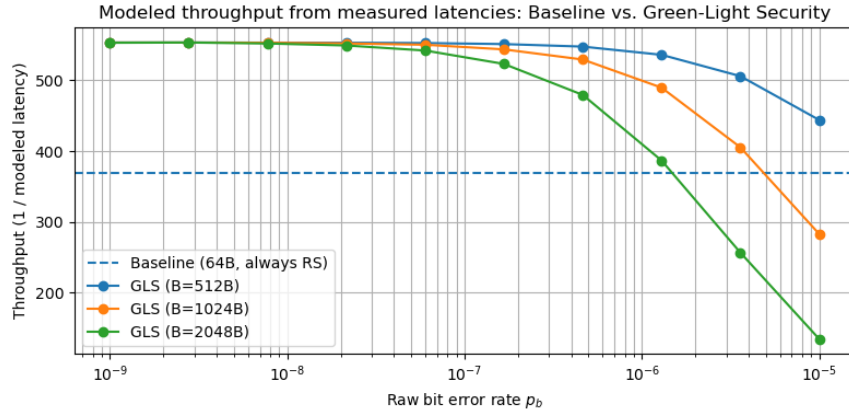


Figure 6: Measured throughput vs. raw bit-error rate for baseline and Green-Light Security configurations.

6 Discussion and Implications

Green-Light Security offers a new perspective on how protection mechanisms should be organized in modern AI memory hierarchies. Rather than treating integrity and reliability as inseparable functions that must accompany every transfer, the proposed framework relocates reliability off the fast path and preserves in-path integrity verification at a coarser granularity. This separation is motivated by the observation that AI workloads naturally operate on large blocks of data and that error rates in high-bandwidth memory systems remain low enough for off-path correction to be invoked only rarely. By matching protection placement to workload characteristics, the system maintains strong correctness guarantees while significantly reducing the steady protection tax.

A key implication of this design is that memory systems need not adhere to a monolithic protection strategy. The results in Section 4 reveal that the optimal protection granularity depends jointly on the raw bit-error rate, the relative bandwidth of the off-path tier, and the available ECC decoding throughput. These dependencies create a “green-light” region of operation in which the fast tier remains bandwidth-bound, leading to both cost reduction and speed improvement. As memory densities increase and process technologies become less reliable, this flexibility becomes increasingly important: instead of strengthening protection uniformly, systems can respond by shifting more work to the off-path reliability engine while keeping in-path latency nearly unchanged.

The broader implications extend beyond memory protection. The same principle (i.e., continuous

lightweight checks backed by heavy corrective action only when necessary) applies to a wide class of AI systems that struggle with the tension between performance and security. Emerging AI accelerators integrate heterogeneous memory, compute-in-memory modules, and near-memory processing units; these architectures face similar trade-offs regarding where to place protection, how frequently to invoke it, and how to amortize metadata. Green-Light Security provides a quantitative framework that unifies reliability, integrity, cost, and throughput, offering a systematic approach to designing these systems.

Finally, the prototype validation demonstrates that the conceptual principles translate cleanly into a physical setting. Despite its simplicity, the prototype follows the same control flow as the full design, exhibits the expected latency asymmetry, and reproduces the transition between HBM-bound and DRAM-bound regimes predicted by the analytical model. This suggests that the architectural principles are robust and suitable for further refinement on more advanced platforms, including FPGA-based prototypes, GPU simulators, and hybrid HBM+DRAM systems.

7 Conclusion

Green-Light Security reorganizes protection across memory tiers to reduce the steady protection tax while preserving strict end-to-end correctness. By keeping integrity verification in-path, moving reliability correction off-path, and enlarging protection granularity to match AI access patterns, the framework improves both performance and capacity relative to conventional monolithic designs. A unified mathematical formulation quantified the interaction among reliability, integrity, cost, and throughput, revealing an operating region where the fast tier remains bandwidth-bound and gains roughly 1.38x throughput. Extensive simulations and a compact hardware prototype validated these predictions, showing that in-path checks incur negligible delay while off-path corrections are rare enough to avoid throttling the system. As AI infrastructures scale, Green-Light Security offers a practical foundation for efficient, strongly protected memory architectures.

References

- [1] C. Silvano, D. Ielmini, F. Ferrandi, L. Fiorin, S. Curzel, L. Benini, F. Conti, A. Garofalo, C. Zambelli, E. Calore, S. Schifano, M. Palesi, G. Ascia, D. Patti, N. Petra, D. De Caro, L. Lavagno, T. Urso, V. Cardellini, G. C. Cardarilli, R. Birke, and S. Perri, “A survey on deep learning hardware accelerators for heterogeneous HPC platforms,” *ACM Computing Surveys*, vol. 57, no. 11, pp. 1–39, 2025.
- [2] Q. Huang, P.-A. Tsai, J. S. Emer, and A. Parashar, “Mind the gap: Attainable data movement and operational intensity bounds for tensor algorithms,” in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, pp. 150–166, IEEE, 2024.
- [3] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2023.
- [4] S. Lin and D. J. C. Jr., *Error control coding - fundamentals and applications*. Prentice Hall, 1983.
- [5] W. Huffman and V. Pless, *Fundamentals of Error-Correcting Codes*. Cambridge University Press, 2003.
- [6] A. J. Menezes, P. C. Van Oorschot, and S. A. Vanstone, *Handbook of applied cryptography*. CRC press, 2018.